

1. SMD + distribución estadística por **label_3m**

Este query cubre los principales features continuos, incluyendo Current y Z challengers. Para los features con availability flag, **solo compara el valor numérico cuando el feature está disponible**, evitando que los 0 imputados/coalesced distorsionen el análisis.

```
feature_smd_summary <- sdf_sql(  
  sc,  
  "  
  WITH base AS (  
  
    SELECT  
      label_3m,  
  
      deterioration_acceleration,  
      adverse_acceleration,  
      max_dpd_jump_recent_3m,  
  
      active_accounts_change_vs_baseline,  
  
      balance_change_model,  
      log1p_abs_balance_change_model,  
      balance_change_available_flag,  
  
      past_due_change_model,  
      past_due_change_available_flag,  
  
      delinquency_share_change_model,  
      delinquency_change_available_flag,  
  
      avg_utilization_change_model,  
      weighted_utilization_change_model,  
      utilization_change_available_flag,  
  
      active_accounts_z_model,  
      active_accounts_z_available_flag,  
  
      avg_utilization_z_model,  
      avg_utilization_z_available_flag,  
  
      weighted_utilization_z_model,  
      weighted_utilization_z_available_flag  
  
    FROM bu_frd.model_features_ab_final_2022_2025  
  ),
```

long_features AS (

SELECT

label_3m,
feature_name,
feature_value

FROM base

LATERAL VIEW STACK(

13,

'deterioration_acceleration',
CAST(deterioration_acceleration AS DOUBLE),
1,

'adverse_acceleration',
CAST(adverse_acceleration AS DOUBLE),
1,

'max_dpd_jump_recent_3m',
CAST(max_dpd_jump_recent_3m AS DOUBLE),
1,

'active_accounts_change_vs_baseline',
CAST(active_accounts_change_vs_baseline AS DOUBLE),
1,

'balance_change_model',
CAST(balance_change_model AS DOUBLE),
CAST(balance_change_available_flag AS INT),

'log1p_abs_balance_change_model',
CAST(log1p_abs_balance_change_model AS DOUBLE),
CAST(balance_change_available_flag AS INT),

'past_due_change_model',
CAST(past_due_change_model AS DOUBLE),
CAST(past_due_change_available_flag AS INT),

'delinquency_share_change_model',
CAST(delinquency_share_change_model AS DOUBLE),
CAST(delinquency_change_available_flag AS INT),

'avg_utilization_change_model',
CAST(avg_utilization_change_model AS DOUBLE),
CAST(utilization_change_available_flag AS INT),

```

'weighted_utilization_change_model',
CAST(weighted_utilization_change_model AS DOUBLE),
CAST(utilization_change_available_flag AS INT),

'active_accounts_z_model',
CAST(active_accounts_z_model AS DOUBLE),
CAST(active_accounts_z_available_flag AS INT),

'avg_utilization_z_model',
CAST(avg_utilization_z_model AS DOUBLE),
CAST(avg_utilization_z_available_flag AS INT),

'weighted_utilization_z_model',
CAST(weighted_utilization_z_model AS DOUBLE),
CAST(weighted_utilization_z_available_flag AS INT)

) s AS feature_name, feature_value, available_flag

WHERE available_flag = 1
AND feature_value IS NOT NULL
),

stats AS (

SELECT
    feature_name,
    label_3m,

    COUNT(*) AS n,

    AVG(feature_value) AS mean_value,
    STDDEV_SAMP(feature_value) AS sd_value,

    PERCENTILE_APPROX(feature_value, 0.05) AS p05,
    PERCENTILE_APPROX(feature_value, 0.25) AS p25,
    PERCENTILE_APPROX(feature_value, 0.50) AS median,
    PERCENTILE_APPROX(feature_value, 0.75) AS p75,
    PERCENTILE_APPROX(feature_value, 0.95) AS p95

FROM long_features

GROUP BY
    feature_name,
    label_3m
),

pos AS (

```

```

SELECT *
FROM stats
WHERE label_3m = 1
),

```

neg AS (

```

SELECT *
FROM stats
WHERE label_3m = 0
)

```

SELECT

p.feature_name,

n.n AS n_label_0,

p.n AS n_label_1,

n.mean_value AS mean_label_0,

p.mean_value AS mean_label_1,

p.mean_value - n.mean_value

AS mean_difference,

n.median AS median_label_0,

p.median AS median_label_1,

n.p05 AS p05_label_0,

p.p05 AS p05_label_1,

n.p25 AS p25_label_0,

p.p25 AS p25_label_1,

n.p75 AS p75_label_0,

p.p75 AS p75_label_1,

n.p95 AS p95_label_0,

p.p95 AS p95_label_1,

SQRT(

(

POWER(n.sd_value, 2)

+

POWER(p.sd_value, 2)

) / 2.0

) AS pooled_sd,

```

CASE
  WHEN SQRT(
    (
      POWER(n.sd_value, 2)
      +
      POWER(p.sd_value, 2)
    ) / 2.0
  ) > 0

  THEN
    (
      p.mean_value - n.mean_value
    )
    /
    SQRT(
      (
        POWER(n.sd_value, 2)
        +
        POWER(p.sd_value, 2)
      ) / 2.0
    )
  END AS standardized_mean_difference

FROM pos p

INNER JOIN neg n
  ON p.feature_name = n.feature_name

ORDER BY ABS(
  CASE
    WHEN SQRT(
      (
        POWER(n.sd_value, 2)
        +
        POWER(p.sd_value, 2)
      ) / 2.0
    ) > 0

    THEN
      (
        p.mean_value - n.mean_value
      )
      /
      SQRT(
        (
          POWER(n.sd_value, 2)
          +
          POWER(p.sd_value, 2)
        )

```

```

    ) / 2.0
  )
END
) DESC
"
) %>%
  collect()

feature_smd_summary

```

De aquí me interesan especialmente:

```

feature_name
mean_label_0
mean_label_1
median_label_0
median_label_1
standardized_mean_difference

```

No vamos a interpretar un SMD pequeño automáticamente como “feature débil”; eso lo contrastaremos con los deciles.

2. Shape analysis por deciles

Aquí excluyo deliberadamente *deterioration*, *adverse* y *max_dpd_jump*, porque tienen muchos valores repetidos y se entienden mejor categóricamente. Esos aparecen en el query 4.

Además, el *lift* aquí se calcula contra el **positive rate del subconjunto donde el feature está disponible**, no contra el 9.09% global. Esto nos permite separar:

numericshape

de:

availabilitysignal

```

feature_decile_shape <- sdf_sql(
  sc,
  "
  WITH base AS (

    SELECT
      label_3m,

```

```

    active_accounts_change_vs_baseline,

    balance_change_model,
    log1p_abs_balance_change_model,
    balance_change_available_flag,

    past_due_change_model,
    past_due_change_available_flag,

    delinquency_share_change_model,
    delinquency_change_available_flag,

    avg_utilization_change_model,
    weighted_utilization_change_model,
    utilization_change_available_flag,

    active_accounts_z_model,
    active_accounts_z_available_flag,

    avg_utilization_z_model,
    avg_utilization_z_available_flag,

    weighted_utilization_z_model,
    weighted_utilization_z_available_flag

FROM bu_frd.model_features_ab_final_2022_2025
),

long_features AS (

    SELECT
        label_3m,
        feature_name,
        feature_value

    FROM base

    LATERAL VIEW STACK(

        10,

        'active_accounts_change_vs_baseline',
        CAST(active_accounts_change_vs_baseline AS DOUBLE),
        1,

        'balance_change_model',
        CAST(balance_change_model AS DOUBLE),
        CAST(balance_change_available_flag AS INT),

```

```
'log1p_abs_balance_change_model',  
CAST(log1p_abs_balance_change_model AS DOUBLE),  
CAST(balance_change_available_flag AS INT),
```

```
'past_due_change_model',  
CAST(past_due_change_model AS DOUBLE),  
CAST(past_due_change_available_flag AS INT),
```

```
'delinquency_share_change_model',  
CAST(delinquency_share_change_model AS DOUBLE),  
CAST(delinquency_change_available_flag AS INT),
```

```
'avg_utilization_change_model',  
CAST(avg_utilization_change_model AS DOUBLE),  
CAST(utilization_change_available_flag AS INT),
```

```
'weighted_utilization_change_model',  
CAST(weighted_utilization_change_model AS DOUBLE),  
CAST(utilization_change_available_flag AS INT),
```

```
'active_accounts_z_model',  
CAST(active_accounts_z_model AS DOUBLE),  
CAST(active_accounts_z_available_flag AS INT),
```

```
'avg_utilization_z_model',  
CAST(avg_utilization_z_model AS DOUBLE),  
CAST(avg_utilization_z_available_flag AS INT),
```

```
'weighted_utilization_z_model',  
CAST(weighted_utilization_z_model AS DOUBLE),  
CAST(weighted_utilization_z_available_flag AS INT)
```

```
) s AS feature_name, feature_value, available_flag
```

```
WHERE available_flag = 1  
AND feature_value IS NOT NULL
```

```
),
```

```
ranked AS (
```

```
SELECT
```

```
*,
```

```
NTILE(10) OVER (  
    PARTITION BY feature_name  
    ORDER BY feature_value  
) AS decile
```

```

    FROM long_features
),

feature_base AS (

    SELECT
        feature_name,

        AVG(
            CAST(label_3m AS DOUBLE)
        ) AS available_positive_rate

    FROM long_features

    GROUP BY feature_name
),

decile_stats AS (

    SELECT
        feature_name,
        decile,

        COUNT(*) AS parties,

        SUM(
            CASE WHEN label_3m = 1
                THEN 1 ELSE 0 END
        ) AS positives,

        SUM(
            CASE WHEN label_3m = 0
                THEN 1 ELSE 0 END
        ) AS negatives,

        MIN(feature_value) AS min_feature_value,
        MAX(feature_value) AS max_feature_value,
        AVG(feature_value) AS avg_feature_value,

        AVG(
            CAST(label_3m AS DOUBLE)
        ) AS positive_rate

    FROM ranked

    GROUP BY
        feature_name,

```

```

        decile
    )

SELECT
    d.feature_name,
    d.decile,

    d.parties,
    d.positives,
    d.negatives,

    d.min_feature_value,
    d.max_feature_value,
    d.avg_feature_value,

    d.positive_rate,

    b.available_positive_rate
    AS feature_available_baseline_rate,

    CASE
        WHEN b.available_positive_rate > 0
        THEN
            d.positive_rate /
            b.available_positive_rate
        END AS lift_vs_available_baseline

FROM decile_stats d

INNER JOIN feature_base b
    ON d.feature_name = b.feature_name

ORDER BY
    d.feature_name,
    d.decile
"
) %>%
collect()

```

feature_decile_shape

Este output debería tener aproximadamente:

$$10 \text{ features} \times 10 \text{ deciles} = 100 \text{ rows}$$

y será el principal insumo para clasificar:

- monotonic;
 - approximately monotonic;
 - U-shaped;
 - nonlinear;
 - flat/weak.
-

3. Availability signal

Este query mide directamente cuánto cambia el outcome según el valor esté disponible o no.

Incluye:

- positive rate available;
- positive rate unavailable;
- rate ratio;
- odds ratio;
- coverage dentro de positives;
- coverage dentro de negatives.

```
feature_availability_signal <- sdf_sql(
  sc,
  "
  WITH flags AS (

    SELECT
      label_3m,
      feature_name,
      CAST(flag_value AS INT) AS flag_value

    FROM bu_frd.model_features_ab_final_2022_2025

    LATERAL VIEW STACK(

      7,

      'balance_change_available',
      balance_change_available_flag,

      'past_due_change_available',
      past_due_change_available_flag,

      'delinquency_change_available',
      delinquency_change_available_flag,

      'utilization_change_available',
```

```

        utilization_change_available_flag,

        'active_accounts_z_available',
        active_accounts_z_available_flag,

        'avg_utilization_z_available',
        avg_utilization_z_available_flag,

        'weighted_utilization_z_available',
        weighted_utilization_z_available_flag

    ) s AS feature_name, flag_value
),

x AS (

    SELECT
        feature_name,

        SUM(
            CASE WHEN flag_value = 1
                THEN 1 ELSE 0 END
        ) AS available_n,

        SUM(
            CASE WHEN flag_value = 0
                THEN 1 ELSE 0 END
        ) AS unavailable_n,

        SUM(
            CASE
                WHEN flag_value = 1
                AND label_3m = 1
                THEN 1 ELSE 0
            END
        ) AS positive_available,

        SUM(
            CASE
                WHEN flag_value = 1
                AND label_3m = 0
                THEN 1 ELSE 0
            END
        ) AS negative_available,

        SUM(
            CASE
                WHEN flag_value = 0

```

```

        AND label_3m = 1
        THEN 1 ELSE 0
    END
) AS positive_unavailable,

SUM(
    CASE
        WHEN flag_value = 0
            AND label_3m = 0
            THEN 1 ELSE 0
        END
) AS negative_unavailable,

SUM(
    CASE WHEN label_3m = 1
        THEN 1 ELSE 0 END
) AS total_positive,

SUM(
    CASE WHEN label_3m = 0
        THEN 1 ELSE 0 END
) AS total_negative

FROM flags

GROUP BY feature_name
)

SELECT
    feature_name,

    available_n,
    unavailable_n,

    positive_available,
    positive_unavailable,

    positive_available * 1.0 /
        available_n
        AS positive_rate_available,

    positive_unavailable * 1.0 /
        unavailable_n
        AS positive_rate_unavailable,

    CASE
        WHEN positive_unavailable > 0
            AND unavailable_n > 0

```

```

THEN
    (
        positive_available * 1.0 /
        available_n
    )
    /
    (
        positive_unavailable * 1.0 /
        unavailable_n
    )
END AS rate_ratio_available_vs_unavailable,

CASE
    WHEN positive_available > 0
    AND negative_available > 0
    AND positive_unavailable > 0
    AND negative_unavailable > 0

    THEN
        (
            positive_available * 1.0 /
            negative_available
        )
        /
        (
            positive_unavailable * 1.0 /
            negative_unavailable
        )
    END AS odds_ratio_available_vs_unavailable,

    positive_available * 1.0 /
    total_positive
    AS pct_positive_with_available,

    negative_available * 1.0 /
    total_negative
    AS pct_negative_with_available

FROM x

ORDER BY
    rate_ratio_available_vs_unavailable DESC
"
) %>%
collect()

feature_availability_signal

```

Este probablemente va a producir algunos de los resultados más fuertes del feature-validation block.

4. Binary, discrete y coverage features

Este query cubre los features para los que los deciles no son la mejor representación.

Incluye:

- `active_accounts_changed_flag`;
- dirección de `active_accounts_change`;
- `deterioration_acceleration`;
- `adverse_acceleration`;
- `max_dpd_jump_recent_3m`;
- número de meses disponibles Recent/Prior para balance;
- past due;
- delinquency;
- utilization.

```
feature_categorical_shape <- sdf_sql(
  sc,
  "
  WITH categorical AS (

    SELECT
      label_3m,
      'active_accounts_changed_flag'
      AS feature_name,

      CAST(
        active_accounts_changed_flag
        AS STRING
      ) AS feature_level

    FROM bu_frd.model_features_ab_final_2022_2025

  UNION ALL

    SELECT
      label_3m,
      'active_accounts_change_direction'
      AS feature_name,
```

```
CASE
  WHEN active_accounts_change_vs_baseline < 0
    THEN 'NEGATIVE'
  WHEN active_accounts_change_vs_baseline > 0
    THEN 'POSITIVE'
  ELSE 'ZERO'
END AS feature_level

FROM bu_frd.model_features_ab_final_2022_2025
```

UNION ALL

```
SELECT
  label_3m,
  'deterioration_acceleration_direction'
  AS feature_name,

  CASE
    WHEN deterioration_acceleration < 0
      THEN 'NEGATIVE'
    WHEN deterioration_acceleration > 0
      THEN 'POSITIVE'
    ELSE 'ZERO'
  END AS feature_level

FROM bu_frd.model_features_ab_final_2022_2025
```

UNION ALL

```
SELECT
  label_3m,
  'adverse_acceleration_direction'
  AS feature_name,

  CASE
    WHEN adverse_acceleration < 0
      THEN 'NEGATIVE'
    WHEN adverse_acceleration > 0
      THEN 'POSITIVE'
    ELSE 'ZERO'
  END AS feature_level

FROM bu_frd.model_features_ab_final_2022_2025
```

UNION ALL

```
SELECT
  label_3m,
  'max_dpd_jump_recent_3m'
  AS feature_name,

  CASE
    WHEN max_dpd_jump_recent_3m = 0
      THEN '0'
    WHEN max_dpd_jump_recent_3m = 1
      THEN '1'
    ELSE '2+'
  END AS feature_level

FROM bu_frd.model_features_ab_final_2022_2025
```

UNION ALL

```
SELECT
  label_3m,
  'balance_recent_months_available',
  CAST(balance_recent_months_available AS STRING)

FROM bu_frd.model_features_ab_final_2022_2025
```

UNION ALL

```
SELECT
  label_3m,
  'balance_prior_months_available',
  CAST(balance_prior_months_available AS STRING)

FROM bu_frd.model_features_ab_final_2022_2025
```

UNION ALL

```
SELECT
  label_3m,
  'past_due_recent_months_available',
```

CAST(past_due_recent_months_available AS STRING)

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

SELECT

label_3m,

'past_due_prior_months_available',

CAST(past_due_prior_months_available AS STRING)

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

SELECT

label_3m,

'delinquency_recent_months_available',

CAST(delinquency_recent_months_available AS STRING)

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

SELECT

label_3m,

'delinquency_prior_months_available',

CAST(delinquency_prior_months_available AS STRING)

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

SELECT

label_3m,

'utilization_recent_months_available',

CAST(utilization_recent_months_available AS STRING)

FROM bu_frd.model_features_ab_final_2022_2025

UNION ALL

SELECT

label_3m,
'utilization_prior_months_available',
CAST(utilization_prior_months_available AS STRING)

FROM bu_frd.model_features_ab_final_2022_2025

),

feature_base AS (

SELECT

feature_name,

AVG(
CAST(label_3m AS DOUBLE)
) AS feature_positive_rate

FROM categorical

GROUP BY feature_name

),

levels AS (

SELECT

feature_name,
feature_level,

COUNT(*) AS parties,

SUM(
CASE WHEN label_3m = 1
THEN 1 ELSE 0 END
) AS positives,

SUM(
CASE WHEN label_3m = 0
THEN 1 ELSE 0 END
) AS negatives,

AVG(
CAST(label_3m AS DOUBLE)
) AS positive_rate

```

FROM categorical

GROUP BY
    feature_name,
    feature_level
)

SELECT
    l.feature_name,
    l.feature_level,

    l.parties,
    l.positives,
    l.negatives,

    l.positive_rate,

    b.feature_positive_rate
    AS baseline_positive_rate,

CASE
    WHEN b.feature_positive_rate > 0

    THEN
        l.positive_rate /
        b.feature_positive_rate

END AS lift

FROM levels l

INNER JOIN feature_base b
    ON l.feature_name = b.feature_name

ORDER BY
    l.feature_name,
    l.feature_level
"
) %>%
collect()

```

feature_categorical_shape

Qué resultados necesito que me pases